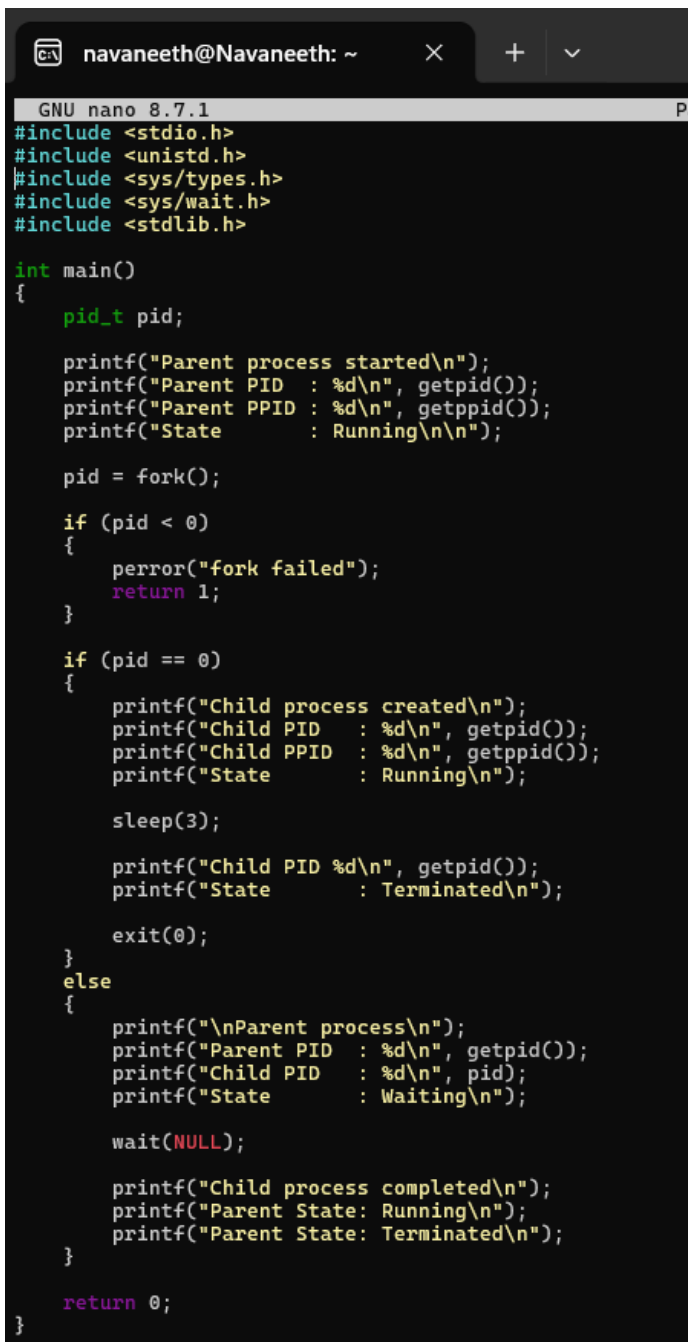


WEEK-2 PRACTICAL

Task 1:

Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID

CODE:

A screenshot of a terminal window with a dark background. The window title is 'navaneeth@Navaneeth: ~'. The terminal shows the GNU nano 8.7.1 editor with a C program. The program includes headers for stdio, unistd, sys/types, sys/wait, and stdlib. The main function starts by printing parent process information (PID, PPID, State: Running), then calls fork(). If fork() fails (pid < 0), it prints an error and returns 1. If fork() succeeds (pid == 0), it enters a child process block where it prints child process information, sleeps for 3 seconds, prints child process information again (State: Terminated), and calls exit(0). If fork() succeeds but it's not the child (else block), it prints parent process information, prints the child's PID, and enters a wait loop (wait(NULL)) until the child process completes. After the wait loop, it prints 'Child process completed' and 'Parent State: Terminated'. Finally, it returns 0.

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());
    printf("State      : Running\n\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process created\n");
        printf("Child PID   : %d\n", getpid());
        printf("Child PPID  : %d\n", getppid());
        printf("State      : Running\n");

        sleep(3);

        printf("Child PID %d\n", getpid());
        printf("State      : Terminated\n");

        exit(0);
    }
    else
    {
        printf("\nParent process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);
        printf("State      : Waiting\n");

        wait(NULL);

        printf("Child process completed\n");
        printf("Parent State: Running\n");
        printf("Parent State: Terminated\n");
    }

    return 0;
}
```

OUTPUT:

```
navaneeth@Navaneeth:~$ gcc ForkProgram.c -o ForkProgram
navaneeth@Navaneeth:~$ ./ForkProgram
Enter a Linux command: shell

Parent Process
Parent PID: 634
Child PID : 635

Child Process
Child PID: 635
Parent PID: 634

Executing command: shell

Command execution failed!

Child process completed.
navaneeth@Navaneeth:~$ ./ForkProgram
Enter a Linux command: ls

Parent Process
Parent PID: 670
Child PID : 673

Child Process
Child PID: 673
Parent PID: 670

Executing command: ls

ForkProgram  ForkProgram.c  'skill 2.c'

Child process completed.
```

Child process completed.

REPORT: I executed the C program successfully. The program accepted a Linux command from the user, created a child process using `fork()`, and executed the command using the `exec()` system call. The parent process waited until the child process finished execution. Finally, the program displayed the process IDs of both the parent and child processes, and the message "Child process completed." was displayed.

Task-2: Using Linux terminal commands (`uname`, `lscpu`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Uname-a:

```
navaneeth@Navaneeth:~$ uname -a
Linux Navaneeth 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 GNU/Linux
navaneeth@Navaneeth:~$
```

REPORT: The `uname -a` command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.

Lscpu:

```
navaneeth@Navaneeth:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:               39 bits physical, 48 bits virtual
Byte Order:                  Little Endian
CPU(s):                      12
  On-line CPU(s) list:       0-11
Vendor ID:                   GenuineIntel
Model name:                  13th Gen Intel(R) Core(TM) i5-1334U
CPU family:                  6
Model:                      186
Thread(s) per core:          2
Core(s) per socket:          6
Socket(s):                   1
Stepping:                    3
BogoMIPS:                    4991.99
Flags:                       fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx f
xsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology
tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid s
se4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervi
sor lahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid
ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushop
t clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnni umip waitpkg gfni vaes vpc
lmulqdq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_lld arch_capabiliti
es
Virtualization features:
  Virtualization:            VT-x
  Hypervisor vendor:         Microsoft
  Virtualization type:       full
Caches (sum of all):
  L1d:                       288 KiB (6 instances)
  L1i:                       192 KiB (6 instances)
  L2:                         7.5 MiB (6 instances)
  L3:                        12 MiB (1 instance)
NUMA:
  NUMA node(s):              1
  NUMA node0 CPU(s):         0-11
Vulnerabilities:
  Gather data sampling:      Not affected
  Ghostwrite:                Not affected
  Indirect target selection: Not affected
  Itlb multihit:             Not affected
  L1tf:                      Not affected
  Mds:                       Not affected
  Meltdown:                  Not affected
  Mmio stale data:           Not affected
  Old microcode:             Not affected
  Reg file data sampling:    Mitigation; Clear Register File
  Retbleed:                  Mitigation; Enhanced IBRS
  Spec rstack overflow:      Not affected
  Spec store bypass:         Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:                Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:                Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI
                             BHI_DIS_S
  Srbds:                    Not affected
  Tsa:                      Not affected
  Tsx async abort:          Not affected
  Vmscape:                  Not affected
navaneeth@Navaneeth:~$ |
```

REPORT: The `lscpu` command displayed the CPU information of my system, such as the processor architecture, number of CPU cores, CPU model, and other hardware details. This helped me understand the processor resources available.

Ps:

```

navaneeth@Navaneeth:~$ ps
    PID TTY          TIME CMD
    302 pts/0        00:00:00 bash
    776 pts/0        00:00:00 ps
navaneeth@Navaneeth:~$ |

```

REPORT: The ps command displayed the processes currently running in my terminal. I observed the process IDs and command names of the active processes.

Top:

```

776 pts/0        00:00:00 ps
navaneeth@Navaneeth:~$ top
top - 04:07:50 up 22 min, 1 user, load average: 0.02, 0.01, 0.00
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7762.6 total, 7190.0 free, 538.3 used, 184.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7224.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    1 root        20   0   24208   15340  11544 S   0.0   0.2   0:00.75 systemd
    2 root        20   0    3180    2208    2072 S   0.0   0.0   0:00.00 init-systemd(Ub
    8 root        20   0    3180    2044    1940 S   0.0   0.0   0:00.00 init
   48 root       19  -1   50396   16776   15488 S   0.0   0.2   0:00.21 systemd-journal
   80 systemd+    20   0   22540   14548   11924 S   0.0   0.2   0:00.08 systemd-resolve
   86 root        20   0   35244   12296    9200 S   0.0   0.2   0:00.46 systemd-udev
  148 root        20   0    2888    1948    1820 S   0.0   0.0   0:00.07 chronyd-starter
  149 root        20   0    4472    3128    2880 S   0.0   0.0   0:00.01 cron
  150 message+    20   0    8820    5312    4644 S   0.0   0.1   0:00.05 dbus-daemon
  154 root        20   0   44092   29760   14616 S   0.0   0.4   0:00.28 networkd-dispat
  158 root        20   0   18428    9420    8180 S   0.0   0.1   0:00.12 systemd-logind
  175 syslog      20   0   220548   5744    4576 S   0.0   0.1   0:00.08 rsyslogd
  236 root        20   0    5492    2748    2564 S   0.0   0.0   0:00.01 agetty
  257 _chrony      20   0   24536   11992    7120 S   0.0   0.2   0:00.30 chronyd
  263 root        20   0   123456   32548   17936 S   0.0   0.4   0:00.17 unattended-upgr
  267 _chrony      20   0   12092    2432    1788 S   0.0   0.0   0:00.01 chronyd
  299 root        20   0    3184    1112     980 S   0.0   0.0   0:00.00 SessionLeader
  300 root        20   0    3200    1248    1108 S   0.0   0.0   0:00.09 Relay(302)
  302 navanee+    20   0    6268    5540    3864 S   0.0   0.1   0:00.11 bash
  303 root        20   0    8648    5572    4728 S   0.0   0.1   0:00.01 login
  364 navanee+    20   0   22508   13432   10984 S   0.0   0.2   0:00.10 systemd
  366 navanee+    20   0   22908    4068    2068 S   0.0   0.1   0:00.00 (sd-pam)
  394 navanee+    20   0    6136    5416    3864 S   0.0   0.1   0:00.02 bash
  788 navanee+    20   0    8216    6076    3924 R   0.0   0.1   0:00.00 top

```

REPORT: The top command displayed the live status of the system. I observed the CPU usage, memory usage, running processes, and system performance updating continuously.